FOCAL для MK61s mini

Версия языка: 17.10.2026. Реализация: транслятор `FOCAL.APP`, общий постраничный редактор `SRCEDIT.APP`, общая байткодная VM. Это описание реализованного языка, а не план разработки.

FOCAL сохраняет калькуляторный ввод, десятичные адреса строк, короткие операторы и доступ к стеку и регистрам МК-61. Редактор, файловый диалог и экранный ввод основаны на общей инфраструктуре TinyBASIC. Предыдущая реализация FOCAL при написании нового транслятора не использовалась.

Запуск и устройство программы

Программа хранится как текст в файловом хранилище. В файл записываются короткие операторы (`A`, `B`, `C` и т. д.); редактор разворачивает их в полные имена, включая `B` → `IF`. Тексты в кавычках и комментарии не изменяются. Перед запуском транслятор разбирает весь текст, сортирует строки и строит образ общей VM. В режиме внешней VM компилятор выгружается до выполнения: в образе остаются числа, адреса и ресурсы, без указателей на память `FOCAL.APP`.

Ненумерованного командного режима в этой версии нет. Ввод, редактирование, сохранение и запуск выполняются через редактор и меню.

```
1.10 ASK N; SET S=0
1.20 FOR I=1,N; SET S=S+I
1.30 PRINT "SUM=",S,!; EXIT
```

Адрес состоит из двух целых частей, разделённых точкой, каждая от 0 до 999; старшая часть должна быть положительной. Это структурный адрес, а не дробное число: `1.1` и `1.10` — разные строки, `01.010` и `1.10` — одна строка. Программная строка обязательно имеет обе части. После адреса нужен пробел. Повторные адреса запрещены.

Несколько операторов в строке разделяются `;`. Точка с запятой внутри текста в кавычках не разделяет операторы. `COMMENT` игнорирует весь остаток строки.

Операторы

Имена нечувствительны к регистру. Поддерживаются полное имя и перечисленная короткая форма; произвольные сокращения не поддерживаются.

Полное имя	Короткая форма	Назначение
`ASK`	`A`	Ввод или ожидание клавиши
`IF`	`B`	Переход по знаку выражения
`COMMENT`	`C`	Комментарий до конца строки
`DO`	`D`	Вызов строки или группы
`EXIT`	`E`	Завершение всей программы
`FOR`	`F`	Строчный цикл
`GOTO`	`G`	Переход на точную строку
`PRINT`	`P`	Вывод чисел и текста
`RETURN`	`R`	Возврат из подпрограммы, возможно со значением
`SET`	`S`	Присваивание
`EXEC`	`X`	Вычисление с отбрасыванием результата
`CLS`	—	Очистка текста и графического кадра
`PSET`	—	Установка, стирание или XOR точки

Полное имя	Короткая форма	Назначение
LINE	—	Отрезок
BOX	—	Контур или заливка прямоугольника
CIRCLE	—	Контур окружности
SHOW	—	Показ накопленного графического кадра
GRAPH	—	График выражения от X

BRANCH, TYPE, QUIT, LET, GOSUB не являются именами операторов этого языка. В частности, BRANCH не оставлен как совместимый алиас.

Переменные и выражения

Числовые переменные: A–Z. a и A — одна переменная. Начальное значение после сброса данных — ноль. Имена A0, AB и строковые переменные отсутствуют. Все обычные переменные глобальны. Данные сохраняются между последовательными запусками в текущем сеансе; пункт «Сброс данных» очищает переменные и массивы именно FOCAL, не массив BASIC.

Допускаются десятичные числа, экспоненциальная запись, PI, круглые скобки, + - * / ^ и унарные +/-. Умножение всегда явное: 2*X.

Приоритет: унарный знак, степень, умножение/деление, сложение/вычитание. Степень связывается справа: 2^3^2 означает 2^(3^2). Например, -2^2 даёт 4; для отрицания результата степени пишите -(2^2). Отрицательный показатель допускается, но запись 2^(-4) нагляднее.

Сравнения и логические операции не входят в выражения FOCAL. Для сравнения двух чисел используйте знак их разности в IF.

Массивы

```
1.10 SET A(0)=12; SET A(1)=24
1.20 SET B(2,3)=A(0)+A(1)
1.30 PRINT B(2,3),!; EXIT
```

Объявление и DIM не нужны. Общий пул содержит 32 элемента всех массивов, поэтому место расходуют только обращённые элементы. Индексы — целые числа от 0 до 32767. Одномерный A(I) соответствует A(I,0). Обычная переменная A и элемент A(0) независимы. Разные имена массивов независимы.

При первом обращении элемент создаётся с нулём, в том числе при чтении. Обращение к новому элементу после заполнения пула — ошибка нехватки места. Дробный или отрицательный индекс — ошибка диапазона.

Стек и регистры МК-61

Стек: .X, .Y, .Z, .T. Память: .R0–.R9, .RA–.RE; .RF доступен только при включённом расширенном режиме памяти.

Ссылки допустимы в выражениях, как получатели SET и ASK:

```
1.10 ASK .X
1.20 SET .R0=.X+1
1.30 PRINT .R0,!; EXIT
```

В качестве счётчика FOR используется только обычная переменная FOCAL.

ASK

```
1.10 ASK "A=",A,"B=",B
1.20 ASK "ELEMENT=",X(3)
1.30 PRINT A+B+X(3),!; EXIT
```

Приглашение применяется к следующему получателю. Без приглашения показывается имя получателя с двоеточием. Значения вводятся последовательно. Ввод может быть выражением, например 2+3, SQRT(2) или A*2. Он тоже транслируется в VM; изменять переменные или вызывать CALL/ARG из такого ввода нельзя.

OK подтверждает ввод; ошибочное выражение позволяет повторить попытку. ESC отменяет ввод и останавливает программу, не присваивая новое значение текущему получателю. Уже введённые предыдущие значения сохраняются. Cx — забой, F+Cx — очистка ввода. Голый ASK ожидает одну клавишу; ESC при этом останавливает программу.

PRINT

Список состоит из выражений, текстовых литералов, ! и формата. Разделитель элементов — запятая; текст можно заключать в двойные или одинарные кавычки. ! завершает текущую строку вывода. Каждый PRINT начинает вывод с первой строки экрана и автоматически выдаёт оставшийся текст в конце оператора. Последовательные PRINT не склеиваются.

```
1.10 PRINT "PI=",%8.3,PI,!
1.20 PRINT "NEXT "; PRINT "LINE",!; EXIT
```

%N.M задаёт ширину поля N от 0 до 64 и число значащих цифр M от 1 до 15, не количество знаков после точки. Число выравнивается справа; слишком длинное число не обрезается. Например, %8.3,PI выдаёт 3.14 с четырьмя пробелами слева. Одиночный % возвращает формат по умолчанию: ширина 0, восемь значащих цифр. Формат действует до конца текущего PRINT.

Между последовательными элементами добавляется один пробел, как в прежнем FOCAL: PRINT 1,2 показывает 1 2. Начальные и конечные пробелы внутри литерала сохраняются. Экранный перенос текста общий с BASIC. Переполнение 96-байтового буфера вывода является ошибкой, не усечением текста.

PRINT AT и графика

FOCAL использует тот же растровый экран, что и TinyBASIC. GW и GH без скобок возвращают ширину и высоту в пикселях: 192×64 для UC1609 и USB Screen, 80×16 для WS0010 с включённой графикой. Если графический экран недоступен, оба значения равны нулю. Например, IF(GW) 9.10,9.10 переводит программу на строку 9.10 при отсутствии графики. Попытка рисования тогда вызывает ошибку NO GRAPHICS.

```
1.10 CLS; BOX 0,0,GW-1,GH-1
1.20 LINE 0,GH-1,GW-1,0; CIRCLE GW/2,GH/2,GH/3
1.30 PRINT AT 2,0,"FOCAL",%5.3,PI
1.40 SHOW; ASK; EXIT
```

Начало координат — левый верхний угол; X растёт вправо, Y — вниз. Координаты округляются до ближайшего целого, половины — от нуля. За пределами экрана рисунок отсекается. Имена новых команд пишутся полностью: старые краткие формы P, S, G, C сохраняют прежнее значение.

Запись	Действие
PSET x,y[,m]	Изменить точку
POINT(x,y)	Прочитать точку: 0 или 1; вне экрана — 0
LINE x1,y1,x2,y2[,m]	Провести отрезок
BOX x1,y1,x2,y2[,m[,f]]	Прямоугольник; f=0 — контур, f=1 — заливка
CIRCLE x,y,r[,m]	Окружность с неотрицательным радиусом
PRINT AT x,y,список	Вывести список PRINT в пиксельной позиции
GRAPH выражение,xmin,xmax,ymin,ymax	Построить график
SHOW	Передать накопленный кадр экрану

m=0 стирает, m=1 рисует, m=2 применяет XOR; по умолчанию m=1. У прямоугольника по умолчанию f=0. CLS очищает весь кадр. Команды меняют буфер; SHOW показывает его. Кадр также показывается перед ASK и при нормальном завершении программы. Голый ASK ожидает клавишу, сохраняя картинку на экране; числовой ASK временно показывает ввод и затем восстанавливает кадр. ESC останавливает программу.

Смена геометрии активного экрана вызывает SCREEN CHANGED.

У PRINT AT координаты и список разделяются запятой: точка с запятой по-прежнему начинает следующий оператор FOCAL. Список сохраняет формат %N.M и пробел между элементами. Символы занимают клетки 6×8: шрифт 5×8 и один пиксель промежутка. Клетки рисуются с непрозрачным фоном, без автоматического переноса. ! переводит вывод на восемь пикселей ниже, к исходной координате X. За краями текст отсекается. Для обновления числа используйте постоянную ширину поля, например PRINT AT 0,0,%6.3,N: пробелы поля стирают оставшиеся цифры. Вложенный CALL сохраняет позицию, накопленный текст и формат вызывающего PRINT.

Обычный PRINT сохраняет прежние правила FOCAL: каждый оператор начинает с первой текстовой строки. При активной графике он перерисовывает только занятые текстовые полосы, сохраняя остальную картинку.

GRAPH вычисляет выражение для каждого столбца и ставит отдельные точки; оси и очистка не добавляются. Границы должны быть конечными, возрастающими. Во время вычисления X временно получает координату столбца; прежнее значение восстанавливается при завершении, ошибке и остановке. Ошибки области определения и деление на ноль пропускают соответствующую точку. CALL и RND внутри выражения графика запрещены; переменные, регистры, массивы, ARG и POINT разрешены. Чтение нового элемента массива сохраняет обычное правило FOCAL: элемент создаётся с нулём.

```
1.10 CLS; GRAPH X*X,-2,2,-1,5
1.20 PRINT AT 2,0,"Y = X^2"; SHOW; ASK; EXIT
```

Полный пример programs/examples/Graphics.foc сам выбирает компоновку для 192×64 или 80×16. Графический буфер не занимает элементы массивов FOCAL. Обновлять прошивку, FOCAL.APP, SRCEDIT.APP, LANGVM.APP, LANGIN.APP и VMSTORE.APP нужно согласованным комплектом; образы VM предыдущей версии пересобираются.

IF и GOTO

```
IF (выражение) отрицательный_адрес,нулевой_адрес,положительный_адрес
```

Адрес выбирается по знаку результата. Любой адрес можно пропустить, сохранив разделяющие запятые. Тогда соответствующая ветвь продолжает остаток строки. Хвостовые адреса можно не писать.

```
1.10 ASK X
1.20 IF(X) 1.40,,; PRINT "NONNEGATIVE",!; EXIT
1.40 PRINT "NEGATIVE",!; EXIT
```

IF сохраняет короткую форму B. GOTO 2.10 требует точный адрес с точкой; GOTO 2 не поддерживается. Адреса являются константами, не выражениями. Не найденный адрес вызывает ошибку при фактическом выполнении перехода.

DO, CALL, ARG и RETURN

DO 2.10 выполняет одну строку и возвращается после её окончания. DO 2 выполняет группу 2.xx и возвращается по её окончании. RETURN нужен для раннего возврата; в конце естественной подпрограммы он необязателен.

Переход GOTO/IF из процедурной группы DO распространяется в вызывающую программу и прекращает этот вызов. Вызов одной строки перехватывает такой переход и возвращается к следующему оператору вызывающей стороны. Внутри функции CALL обычные внутренние переходы сохраняют её параметры и ожидание возвращаемого значения.

Вызов может передать до четырёх числовых аргументов:

```
1.10 DO 2,7,9; EXIT
2.10 PRINT ARG(1)+ARG(2),!; RETURN
```

ARG(1) — первый аргумент текущего вызова. Параметры только читаются; вложенный вызов сохраняет параметры вызывающей подпрограммы. ARG вне вызова или с несуществующим номером — ошибка. Обычные переменные не становятся локальными.

CALL(адрес,аргументы...) выполняет ту же подпрограмму из выражения. Она обязана завершиться явным RETURN выражение:

```
1.10 SET X=3+CALL(2,7,9)*2; PRINT X,!; EXIT
2.10 RETURN ARG(1)+ARG(2)
```

Результат — 35. Для процедурного DO выражение у RETURN вычисляется, но результат отбрасывается. Для CALL обычный RETURN или автоматический возврат без значения — ошибка. Значение последнего случайно вычисленного выражения не используется как результат функции.

Рекурсивный пример:

```
1.10 ASK N; PRINT CALL(2,N),!; EXIT
2.10 IF(ARG(1)-1) 2.30,2.30; RETURN ARG(1)*CALL(2,ARG(1)-1)
2.30 RETURN 1
```

Глубина вызовов ограничена восемью, включая процедурные вызовы. EXIT завершает всю программу, даже из подпрограммы.

FOR

```
FOR I=начало,конец; тело
FOR I=начало,шаг,конец; тело
```

Порядок трёх параметров сохранён: начало, шаг, конец. Без шага он равен +1, не выбирается автоматически. Отрицательный шаг разрешён, нулевой — ошибка. Диапазон включает конечное значение. Начало, шаг и конец вычисляются один раз.

Всё после первого ; — тело цикла, включая несколько операторов:

```
1.10 SET S=0
1.20 FOR I=1,2,9; SET S=S+I; PRINT I,!
1.30 PRINT "SUM=",S,!; EXIT
```

Вложенный FOR также охватывает свой остаток строки. NEXT не нужен. Для большого тела вызывайте группу через DO. IF/GOTO, выходящий из диапазона тела цикла, завершает соответствующий цикл. RETURN возвращается из текущей подпрограммы, а EXIT завершает всю программу.

Функции

```
SIN(X) COS(X) TG(X)        ASIN(X) ACOS(X) ATG(X)
LN(X) LG(X) EXP(X)         SQRT(X) ABS(X)
INT(X) FRAC(X) ROUND(X)    SGN(X)
MAX(X,Y) MIN(X,Y) MOD(X,Y)
RND() PI
CALL(адрес,аргументы...) ARG(позиция)
```

Математика и числовая семантика общие с VM/BASIC и выбранным калькуляторным бэкендом. Тригонометрия учитывает текущий режим МК-61: радианы, градусы или грады. RND() использует выделенный поток FOCAL общего генератора устройства.

MOD — остаток от целочисленного деления с семантикой общей VM. MIN реализован через существующие NEG и MAX, без отдельной инструкции. Нет алиасов TAN, ATAN, LOG, SQR, INV, EXP10; используйте TG, ATG, LG, X^2, 1/X, 10^X.

Редактор и клавиатура

Редактор использует тот же экранный редактор и файловые диалоги, что BASIC. На двухстрочном дисплее обе строки заняты программой; на графическом — число строк определяется настройкой экрана. Горизонтальная прокрутка, SMS-ввод, показ курсора и работа с M8 общие с BASIC.

Полное имя оператора вставляется в поле после номера строки или после ;. Префикс K принудительно вставляет оператор и вне этого поля, если сочетание не занято общей таблицей пунктуации. Если после вручную введённого номера нет пробела, он добавляется автоматически.

Оператор	Клавиша
ASK	a
IF	b или K+b

Оператор	Клавиша
COMMENT	c
DO	К+Сх
EXIT	е или К+е
FOR	× или К+×
GOTO	Г или К+Г
PRINT	Р / радианы или К+Р
SET	х→П или К+х→П
RETURN	В/О

Общая таблица знаков имеет приоритет над операторными сочетаниями: К+а — апостроф, К+с — кавычка, К+В/О — ;, К+ПП — запятая, К++ — =, К+С/П — !, К+←/К+→ — скобки. CLS и EXEC можно набрать через SMS; для EXEC доступна запись X.

Действие	Клавиша
Курсор внутри строки	← / →
Предыдущая / следующая строка	ШГ← / ШГ→
Новая строка	ОК
Сохранение и выход	MENU/ESC
Забой	Сх
Очистить строку; удалить уже пустую	F+Сх

Имена операторов вместе с последующими пробелами — атомарные элементы: стрелки не оставляют курсор внутри полного имени, а забой непосредственно после оператора удаляет его целиком. F+← не является забоем.

Цифры вводятся непосредственно. Латинские буквы — через К и SMS-группы: 8=ABC, 9=DEF, 4=GHI, 5=JKL, 6=MNO, 1=PQRS, 2=TUV, 3=WXYZ. Повтор цифровой клавиши переключает букву. 7/ПП завершает SMS и вводит пробел; 0 завершает SMS без добавления символа.

Калькуляторные макросы работают с выражением слева от курсора, включая текущий аргумент вложенного вызова и выражение после ;:

Сочетание	Преобразование
F+×	квадрат
F+÷	обратное число
F+0	10^X
F+−	SQRT(X)
F+/-/	ABS(X)
F+1, F+2, F+3	EXP, LG, LN
F+4, F+5, F+6	ASIN, ACOS, ATG
F+7, F+8, F+9	SIN, COS, TG

Если слева нет выражения, F+0…F+9 вставляет соответственно ! @ # $ % ^ & * ( ). Эти знаки не обязательно имеют смысл в языке: таблица общая с BASIC. Составное выражение макрос заключает в скобки, например A+B превращается в (A+B)^2, 1/(A+B) или SQRT(A+B). В поле имени файла операторные клавиши и макросы не применяются.

Ограничения и сборка

- До 65 534 байт исходного текста M8. Отдельного предела числа строк нет.
- 32 элемента массивов, индексы 0…32767.
- До четырёх параметров вызова, восьми вложенных вызовов.
- До 16 вложенных циклов; стек выражений VM — 96 числовых ячеек.
- Вводимое выражение — до 64 байт.
- Числовые строки вывода собираются в буфере на 96 байт.

Новый FOCAL собирается как compiler APP и требует согласованную общую VM. По умолчанию при включённом FOCAL выбираются `FOCAL.APP`, `BASIC.APP`, `SRCEDIT.APP`, `LANGVM.APP`, `LANGIN.APP`; альтернативно VM может быть встроена в resident. В `tools/apps/portable/build_portable_app.py --system focal` режим compiler APP включается автоматически. Математический бэкенд выбирается у общего исполнителя. На F401 с FOCAL по умолчанию используется CORE. F411 использует BINARY64 по умолчанию; профиль можно переопределить явно. BINARY64 (`MK61_MATH_BACKEND=0`, прежнее название LIBM) в штатной самостоятельной сборке использует своё `compact_math`, а не внешнюю libm. CORE использует точность и скорость ядра МК-61.

Байткод LBV2 версии 14 и запросы VM версии 13 требуют пересборки согласованного комплекта. Старые образы и несовместимые снимки состояния отклоняются.

В эту версию не входят прямой командный режим, строковые переменные, пользовательские функции с именами, вычисляемые адреса переходов, `WHILE`, блочный `FOR/NEXT`, файловые каналы, самомодификация, графика, обработчики ошибок и событий. Сохранение программы выполняет редактор.

Проверки языка и клавиатуры: `bash tests/run_focal_tests.sh`. Общая VM, ресурсы и BASIC: `bash tests/run_language_vm_tests.sh`.

Исходник читается через окно в 256 байт; целый файл и его развёрнутая копия в RAM не хранятся. Редактор разворачивает названия операторов только в текущем окне, а сохранение потоково возвращает короткую запись команд. Крупные файлы занимают несколько секторов C9. Предел 65 534 связан с 16-битными длиной и смещениями ABI. Индекс строк и байткод также хранятся во Flash и читаются частями; отдельной квоты байткода нет. Его адреса, переходы и сохранённые состояния вызовов используют 32 бита. Размер программы ограничен свободным местом носителя.

Общий `/System/SRCEDIT.APP` использует две банки страниц по 512 байт в журнале Flash. Смена окна фиксирует изменения в черновике; до сохранения исходный файл остаётся прежним. Отмена или неудачная запись не заменяют его. Уже сохранённые страницы черновика используются повторно: следующая правка перезаписывает только изменившиеся страницы, сохраняя прежние до завершения записи. Обновлять прошивку и все языковые APP нужно одним согласованным комплектом.

API, сохранение и диагностика

`CompileFocal(const char*)` проверяет текст, создаёт и сохраняет новую программу с именем FOCAL0 (или следующим свободным индексом), затем выбирает её для запуска. `RunFocal(index)` возвращает FocalRunStatus, включая ошибки компиляции, выполнения и остановку. В русском интерфейсе автоматические имена FOCAL0 показываются как ФОКАЛ0.

Сохранение не требует корректного синтаксиса: незавершённый черновик можно оставить в файле. Сообщение «сохранён» не означает, что программа уже проверена. Имя, коллизии и номер слота проверяются отдельно; неудачная запись или удаление при переименовании не публикует новые данные слота. Editor-view отделён от сохранённого исходника, поэтому отмена редактирования сохраняет прежнюю программу.

Сохранены коды NAME?, SLOT?, MK? и остальные сообщения на русском и английском. При включении MK61_FOCAL_TRACE выводятся сообщения LOAD/SAVE/RUN/ERROR и EXEC в Serial. Для согласованного отладочного bundle задайте `MK61_FOCAL_TRACE=1`; standalone builder принимает `--focal-trace` для FOCAL и LANGVM. В resident используется опциональный DEBUG_IO-сервис; на старом resident без этого capability отладочный APP безопасно пропускает журнал.